

NetRadiant-custom: Modern Lighting Pipeline Plan

Direct PBR, shadow mapping, and SH indirect lighting for a Silent Hill 1 style game

Davis Wen

16 September 2026

1 Goal

Extend NetRadiant-custom and its build pipeline so a level designer can author, preview, and compile maps with a glTF-style metallic-roughness material model, physically based direct lighting with shadow maps, and baked indirect lighting stored as spherical harmonics (SH). The target game is a Silent Hill 1 style third-person title running on an ioquake3-derived engine: a fog-limited town of watertight streets under a low virtual sky, with watertight interiors.

The plan is staged so that every stage ends with something a level designer can look at, and no stage requires undoing the previous one.

2 Guiding decisions

Keep q3map2's BSP and VIS stages untouched.

CSG, the BSP tree, portals, leak detection, t-junctions, patch tessellation, fog volumes, collision lumps, and the file formats are exactly what we do not want to rewrite.

Replace the light stage with a separate tool.

The existing `sh-baker` (Embree, tinyglTF) gains a BSP reader and becomes the light stage. `q3map2's -light` is used only to allocate lightmap UVs, or dropped once the baker atlases on its own.

No BSP format change for thick walls.

The BSP brush and brushside lumps keep every plane of every solid brush, including void-facing planes whose draw surfaces `q3map2` strips. The engine reconstructs closed convex hulls from those lumps at load and uses them as front-face-culled shadow casters.

Editor preview of direct lighting is the fast iteration loop.

Editor preview of baked SH is optional and last, because it needs lightmap UVs on source geometry, which is a new subsystem.

Lock units and colour spaces in Stage 1.

Every later comparison against the engine depends on light units, exposure, and sRGB handling being final in the benchmark map.

3 Current state of the editor (relevant facts)

- Gamepacks are data: `gamepacks/games/*.game` plus a matching directory. The editor scans them at start; no code change is needed to add a game.
- Material parsing is compiled into `plugins/shaders`. Three hard-coded languages exist (Quake 3, Doom 3, Quake 4), selected by the `shaders` attribute of the `.game` file. The Doom 3/Quake 4 path already resolves diffuse, bump, and specular textures per material.

- The camera has a lighting draw mode that renders one additive GLSL pass per light (`radiant/renderstate.cpp`, `GLSLBumpProgram`) using map light entities as omni lights with attenuation textures. Brush and patch vertices already carry tangent and bitangent.
- Rendering goes straight to the 8-bit window buffer; there is no framebuffer object in the camera path. The context is created by `QOpenGLWidget` with no explicit version request.
- The editor has no notion of lightmap UVs; `q3map2`'s light stage allocates them.
- The build menu is a per-gamepack XML list of command lines; each `q3map2` stage is a separate invocation that reads and writes the BSP.

4 Stage 1: Direct light and PBR materials

Outcome. A gamepack, a material format, light entity definitions, and a lit editor preview good enough to author the first benchmark map with final storage formats.

4.1 1.1 Material format and parser

- Define a text material format modelled on the glTF metallic-roughness schema, brace-compatible with the Doom 3 parser: `basecolor`, `normal`, `metallicroughness`, `occlusion`, `emissive`, plus `metallicFactor`, `roughnessFactor`, `emissiveFactor`, `emissiveStrength`, alpha mode and cutoff, double-sided.
- Keep `qer_editorimage`, `surfaceparm`, and `q3map_` keys so `q3map2` and the editor's brush classification keep working. Decide whether the baker reads the same files.
- Add a fourth shader language: new `parsePBR` on `ShaderTemplate`, a new `SHADERLANGUAGE_` value, a module in `plugins/shaders/plugin.cpp` setting the directory and extension, and a `shaders="pbr"` value for the `.game` file.
- Extend `IShader` in `include/ishaders.h` with the extra textures and factors.
- Colour space: base colour and emissive are sRGB and decoded in the shader; normal, metallic-roughness, and occlusion are linear. Check the texture preference paths (gamma, compression) so the loaders do not alter these.

4.2 1.2 Light entities and units

- Entity definitions for point, spot, and sun lights with the keys the engine will read: colour, intensity in physical units, radius, cone angles, and shadow flags.
- The sun lives on `worldspawn` or a dedicated entity, so Stage 2 can read it without a format change. Direction and colour also feed `q3map_sun` for compatibility.
- Plumb the new keys through the entity plugin into `RendererLight` (origin, colour, radius exist; add direction, cone, intensity).

4.3 1.3 Float framebuffer and tonemap

- Add a half-float colour target plus depth in the camera path, with a resolve pass that applies exposure and a tonemap. Physical light units clip in an 8-bit buffer before tonemapping can happen.
- Pulled forward from Stage 2 because shadow maps need framebuffer objects anyway.

4.4 1.4 Lighting program

- New GLSL program replacing Blinn-Phong: GGX distribution, Smith geometry, Schlick Fresnel, Disney diffuse, ported from `sh-renderer/glsl/radiance.frag`.

- Keep the editor’s per-light additive pass structure. Do not port Forward+ tiling; the editor already culls lights per surface on the CPU.
- Base pass draws emissive plus a constant ambient. Point and spot lights accumulate on top.

4.5 1.5 Gamepack and benchmark map

- Gamepack: `.game` file, entity definitions, default build menu, shader list.
- Benchmark map: one street block with two building interiors, a low sky ceiling, a fog volume, a sun, several point and spot lights, and materials exercising every map type.
- Exit criterion: the map looks the same in the editor lighting mode and in the engine’s direct-light-only path, within exposure tolerance.

5 Stage 2: Shadow mapping

Outcome. Sun and spot shadows in the editor preview, cast by closed brush volumes, and a proof in the engine that hull reconstruction from the BSP gives the same result.

5.1 2.1 Caster pass

- Depth-only program drawing all brush faces with front-face culling. Because every brush is a closed convex volume in the editor, the far side of each wall becomes the caster and the effective bias is the wall thickness (second-depth technique). No hand-tuned bias per map.
- Cache the shadow maps and invalidate on scene change or light change; the editor scene changes rarely, so per-frame cost is only sampling.

5.2 2.2 Sun cascades, then spot atlas

- Cascaded shadow maps for the sun first, since a low sky ceiling makes the thin-wall question hardest there. Then the spot light shadow atlas.
- Port `ComputeShadow`, `ComputeSunShadow`, and the Poisson PCF from `radiance.frag` unchanged once the samplers and cascade uniforms exist.

5.3 2.3 Thin receivers

- Patches and models are thin in the editor too. Use normal-offset bias for them, or make them receivers only.

5.4 2.4 Engine-side hull reconstruction (validation)

- In `ioquake3`, rebuild each collision brush’s convex hull from the brush and brushside lumps at load, and use the hulls as the front-face-culled caster mesh. Draw surfaces remain the receivers.
- Known gaps: brushes with non-solid shaders are dropped from the collision lumps. If such a brush must cast, mark its material so the compiler keeps it, or emit it separately. `q3map2`’s bevel planes are redundant on the polytope and do not disturb the intersection.
- Exit criterion: the benchmark map’s shadows match between editor and engine.

6 Stage 3: Lightmap UVs, SH indirect, and the compiled format

Outcome. Baked SH indirect lighting in the engine via a light stage replacement, with no change to the BSP format.

6.1 3.1 Baker as the light stage

- Add a BSP reader to **sh-baker**: draw surfaces, lightmap UVs, materials, and brush hulls for casters. Entities for lights and sun.
- Pipeline: **q3map2 -meta**, **q3map2 -vis**, **q3map2 -light -fast** for UV allocation only, then **sh_baker** in place of the real light stage.
- Later, optionally atlas in the baker and write lightmap coordinates into the draw surfaces directly, dropping the **-light** step.

6.2 3.2 Output and engine loading

- External lightmap images (already supported by the engine) for the L0 band, plus a side file with the three packed SH textures matching **GetLightmapTexel** in **radiance.frag**.
- Engine loads the side file and runs the indirect BRDF. No BSP lump changes.

6.3 3.3 Editor preview of baked SH (optional, last)

- Requires lightmap UVs on source brush faces and patches that match the compiled atlas, either by matching surfaces back to the BSP or by an editor-side atlas (xatlas or similar).
- Only worth doing if designers need indirect preview inside the editor; the engine and the existing **sh-renderer** viewer cover it otherwise.

7 World-building conventions for the town

These are authoring rules, not code, but they determine whether the visibility system works.

- **Low sky ceiling just above the camera.** VIS treats sky as opaque, so the town becomes a network of corridors walled by buildings, and the PVS behaves like an indoor level.
- **Building shells are structural brushes.** Only structural brushes block VIS. Ornament is detail. Props, pipes, and clutter are models.
- **Fog colour equals sky colour.** Buildings end at the ceiling; with matched colours both fade to the same value and the seam disappears. Far plane set at fog opacity distance.
- **Hint brushes across long streets** at intervals so far clusters drop out; area portals in doors between street and interiors.
- **Clamp camera pitch** so the ceiling is never inspected from below.
- **Set worldspawn _blocksize** to zero or a large value for the flat town to avoid pointless axial splits.

8 Risks

Risk	Mitigation
Light units or colour space differ between editor and engine	Fixed in Stage 1.5 by comparing the benchmark map; treat as a blocking exit criterion.
Texture loader preferences alter colour data	Audit gamma and compression paths before choosing where linearisation happens.
Non-solid brushes needed as casters are dropped from collision lumps	Material key to keep them, or a small editor-side caster export.
Long straight streets defeat the PVS	Fog far plane plus hint brushes; accept that the fog is the real culler there.
Editor SH preview is expensive	Scoped optional and last; not on the critical path.
Qt GL context lacks needed features for FBO and half-float	Request an explicit 3.3 compatibility profile in <code>QSurfaceFormat</code> ; the legacy code keeps working.

9 Dependency summary

Item	Depends on	Note
1.4 Lighting program	1.1, 1.2, 1.3	Needs materials, light keys, and a float target.
1.5 Benchmark map	1.1–1.4	Locks units and formats.
2.1 Caster pass	1.3	Reuses FBO plumbing.
2.2 Cascades	1.2	Needs the sun definition.
2.4 Engine hulls	1.5	Needs the benchmark map to compare against.
3.1 Baker	1.1, 1.5	Reads the material format; validated on the benchmark map.
3.2 Engine SH loading	3.1, 2.4	Direct and indirect combine in the engine’s radiance shader.
3.3 Editor SH preview	3.2	Optional.